



CS4002 Applied Programming

National University of Computer & Emerging
Sciences
Islamabad, Pakistan

Ms. Umarah Qaseem



1

Object Oriented Programming - IV

3

3

Inheritance and Polymorphism

8

Inheritance – Introduction

Perhaps the **most powerful feature of OOP**

New classes are **created from existing classes**

New classes absorb all features of existing classes including their data and functions.

Also **enhance** them by **adding their own new features** in form of **new data members** and **new member functions**

14

Inheritance – Terminology

Existing classes are called **base classes**

New classes are called **derived classes**

Objects of derived classes are more specialized as compared to objects of their base classes

15

Inheritance Limitations in C++

All data members (but not private) and **member functions** of base class are inherited to derived class

Constructors, destructors and **= operator** are **not inherited**

16

Base Classes and Derived Classes

A **class** can be **derived** from one or more than one classes, (single as well as multiple base classes).

Syntax

To define a **derived class**, we use a **class derivation list** to specify the base class(es):

*class **derived-class**: **access-specifier** **base-class***

Where **access-specifier** is one of **public**, **protected**, or **private**.

NOTE: *If the access-specifier is not used, then it is private by default.*

17

Constructor and Destructor Chaining

Constructors are **invoked** in the order of the following chain (**inheritance chain**):

Derived Class → Base Class → Base Class

and **executed** in the order of the following chain:

Derived Class ← Base Class ← Base Class

A **base class constructor** is **executed before** the **derived class constructor**.

Conversely, the **destructors are automatically invoked+executed in reverse order**, with the **derived class's destructor invoked first**.

Derived Class → Base Class → Base Class

This is called **constructor and destructor chaining**.

18

Constructing and Destructor Chaining - Example

<pre>class Person { public: Person() { cout << "Person's constructor \n" ; } ~Person() { cout << "Person's destructor \n" ; } };</pre>	<pre>void main() { Faculty faculty; }</pre>
<pre>class Employee: public Person{ public: Employee() { cout << "Employee's constructor \n" ; } ~Employee() { cout << "Employee's destructor \n " ; } };</pre>	
<pre>class Faculty: public Employee{ public: Faculty() { cout << "Faculty's constructor \n " ; } ~Faculty() { cout << "Faculty's destructor \n"; } };</pre>	

19

Explicitly Invoking Base Class Constructors

Unlike data fields and functions, the constructors of a base class are not inherited in the derived class.

Base class constructors can only be invoked from the constructors of the derived classes to initialize the data fields in the base class. The syntax to invoke it is as follows:

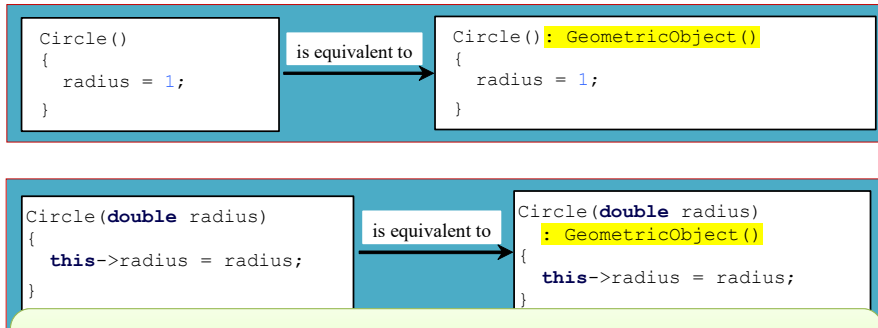
```
DerivedClass(parameterList): BaseClass()
{ // Perform initialization }
```

```
DerivedClass(parameterList): BaseClass(argumentList)
{ // Perform initialization }
```

22

No-Arg Constructor in Base Class

A constructor in a derived class must always invoke a constructor in its base class. If a **base constructor is not invoked explicitly, the base class's no-arg constructor is invoked by default**. For example,



Constructors of ONLY direct base class can be invoked

23

The *protected* Keyword

- ❑ A ***protected*** data field or a protected function in a base class can be accessed by name in its derived classes (*not with the object of the derived class*).

24

The *protected* Keyword

- ❑ A *protected member* is *accessible from within the class in which it is declared, and from within any class derived from the class that declared it.*
- ❑ A **protected member** of a **base class** is **accessible** in a **derived class only** (not with the object of the derived class)

25

Find Errors

<pre> class Base{ protected: int y; private: int x; public: int z; Base() { x = 0; y = 0; z = 0; } }; class Derived: public Base{ public: void showX(){ cout<<"x = "<<x; } void showY(){ cout<<"y = "<<y; } void showZ(){ cout<<"z = "<<z; } }; </pre>	<pre> void main() { Derived d; cout<<d.x; cout<<d.y; cout<<d.z; } </pre>
--	--

26

Find Errors

<pre> class Base{ protected: int y; private: int x; public: int z; Base() { x = 0; y = 0; z = 0; } }; class Derived: public Base{ public: void showX(){ /*cout<<"x = "<<x;*/} void showY(){ cout<<"y = "<<y; } void showZ(){ cout<<"z = "<<z; } }; </pre>	<pre> void main() { Derived d; //cout<<d.x; //cout<<d.y; cout<<d.z; } </pre>
---	--

27

Types of Inheritance specified by Access Specifiers

Public Inheritance: When deriving a class from a public base class, **public members of the base class become public members of the derived class** and **protected members of the base class become protected members of the derived class**. A base class's private members are never accessible directly from a derived class, but can be accessed through calls to the public and protected members of the base class.

Protected Inheritance: When deriving from a protected base class, **public and protected members of the base class become protected members of the derived class**.

Private Inheritance: When deriving from a private base class, **public and protected members of the base class become private members of the derived class**.

29

Types of Inheritance specified by Access Specifier

```

class A {
    public:      int  x;
    protected: int  y;
    private:    int  z;
};
class B : public A
{
    // x remains public and accessible here
    // y remains protected and accessible here
    // z remains private and is not accessible here
};
class C : protected A
{
    // x becomes protected and accessible here
    // y remains protected and accessible here
    // z remains private and is not accessible here
};
class D : private A
{
    // x becomes private, accessible here but not in subclasses of class D
    // y becomes private, accessible here but not in subclasses of class D
    // z remains private and is not accessible here
};

```

Public, Private, and Protected Inheritance

30

Function Overloading and Function Overriding

Overloading a function is a way to provide more than one function with the same name but with different signatures to distinguish them.

In **function overriding** (redefine a function), the function must be defined in the derived class using the same signature and same return type as in its base class.

32

Virtual Functions

A **virtual function** is a **member function** that you expect to be redefined in derived classes.

When you refer to a **derived class object** using a **pointer to the base class**, (you can call a virtual function for that object and execute the derived class's version of the function).

Virtual functions ensure that the **correct function** is called for an object

The keyword **virtual** tells compiler that *it should not perform early binding*

33

Function Overriding and Polymorphism

In C++, **redefining a virtual function** in a derived class is called **overriding a function**.

Polymorphism is a **mechanism** that allows you to implement a function in different ways

One of the key features of derived classes is that a **pointer to a derived class** is **type-compatible** with a pointer to its base class.

34

Need for Polymorphism – Example (1)

<pre> class Base{ public: void show() { cout<<"Base\n"; } }; class Derv1:public Base{ public: void show() { cout<<"derv1\n"; } }; class Derv2:public Base{ public: void show() { cout<<"derv2\n"; } }; </pre>	<pre> void main() { Base b; Derv1 d1; Derv2 d2; b.show(); d1.show(); d2.show(); b = d1; b.show(); b = d2; b.show(); } </pre>
--	---

35

Need for Polymorphism – Example (2)

<pre> class Base{ public: void show() { cout<<"Base\n"; } }; class Derv1:public Base{ public: void show() { cout<<"derv1\n"; } }; class Derv2:public Base{ public: void show() { cout<<"derv2\n"; } }; </pre>	<pre> void main() { Base* b; Derv1 d1; Derv2 d2; b = &d1; b->show(); b = &d2; b->show(); } </pre>
--	---

36

Polymorphism

```

class Base{
public:
    virtual void show()
    { cout<<"Base\n";    }
};

class Derv1:public Base{
public:
    void show()
    { cout<<"derv1\n";    }
};

class Derv2:public Base{
public:
    void show()
    { cout<<"derv2\n";    }
};

void main()
{
    Base*   b;
    Derv1   d1;
    Derv2   d2;

    b = &d1;
    b->show();

    b = &d2;
    b->show();
}

```

37

Define Virtual Functions – Enable Dynamic Binding

To enable dynamic binding for a function, you need to do two things:

1. The function must be declared **virtual** in the base class.
2. The variable that references the object for the function must **contain the address of the object**.

Use **pointer** of **base class** to hold **address of derived class objects**.

44

No Dynamic Binding with Reference Variable

<pre> class Base{ public: virtual void show() { cout<<"Base\n"; } }; class Derv1:public Base{ public: void show() { cout<<"derv1\n"; } }; class Derv2:public Base{ public: void show() { cout<<"derv2\n"; } }; </pre>	<pre> void main() { Base b; Derv1 d1; Derv2 d2; b.show(); b=d1; b.show(); b=d2; b.show(); } </pre>
--	---

45

Note:

If a function is defined *virtual* in a base class, it is automatically *virtual* in all its derived classes. It is **not necessary to add the keyword *virtual* in the function declaration in the derived class.**

46

Note:

Once virtual, always virtual through the inheritance chain.

If the grandchild class doesn't provide its own version of virtual function, then the version from its nearest ancestor (Derived) will be used.

Classes are: Base, Derived, GrandChild

47

When to use virtual functions?

- ❑ If a function defined in a base class needs to be redefined in its derived classes, **you should declare it *virtual* to avoid confusions and mistakes.**
- ❑ On the other hand, if a function will not be redefined, ***it is more efficient without declaring it virtual***, because ***it takes more time and system resource to bind virtual functions dynamically at runtime.***

49

Abstract Classes and Pure Virtual Functions

- In the **inheritance hierarchy**, classes become more **specific** and **concrete** with *each new derived class*.
- If you **move from a derived class back** up to its parent and ancestor classes, the **classes become more general** and less specific.
- Class design should ensure that a **base class contains common features** of its **derived classes**.
- Sometimes a **base class is so abstract that it cannot have any specific instances**. Such a class is referred to as an **abstract class**.
- For a **class to be an abstract class**, it must contain at least one pure virtual function.

50

Abstract Classes and Pure Virtual Functions

- **Pure virtual function** (e.g., Show()) can be declared as:

virtual void show() = 0;

- Once you've placed a pure virtual function in the base class, **you must override it in all derived classes** from which you want to instantiate objects.
- If a class does not override the pure virtual function, **it becomes an abstract class itself**.
- It is **not possible to instantiate object of abstract class**.

51

Abstract class and Pure Virtual Function - Example

```

class Base{
public:
    virtual void show() = 0;
};
class Derv1:public Base{
public:
    void show()
    {
        cout<<"derv1\n";
    }
};
class Derv2:public Base{
public:
    void show()
    {
        cout<<"derv2\n";
    }
};

void main(){
    // Base bs;

    Base *ptr;
    Derv1 d1;
    Derv2 d2;

    ptr = &d1;
    ptr->show();

    ptr = &d2;
    ptr->show();
}

```

52

Abstract class with Non-Virtual Functions and Members

```

class Base{
public:
    virtual void show() = 0;
};
class Derv1:public Base{
public:
    void show()
    {
        cout<<"derv1\n";
    }
};
class Derv2:public Base{
public:
    void show()
    {
        cout<<"derv2\n";
    }
};

void main(){
    // Base bs;

    Base *ptr;
    Derv1 d1;
    Derv2 d2;

    ptr = &d1;
    ptr->show();

    ptr = &d2;
    ptr->show();
}

```

53

Virtual Destructors

- ❑ In C++, **classes** and **class destructors** can be **virtual** but **not constructors**.
- ❑ If a **base class destructor** is **not virtual** then ***delete***, like a **normal member function**, **calls the destructor** for the **base class**, **not the destructor** for the **derived class**.

54

Virtual Destructors

<pre> class Base { public: ~Base() { cout<<" Base Destroyed \n"; } }; class Derv: public Base { public: ~Derv() { cout<<" Derv Destroyed \n"; } }; </pre>	<pre> void main() { Base *p = new Derv; delete p; /*will call destructor of base class not the derived class*/ } </pre>
--	--

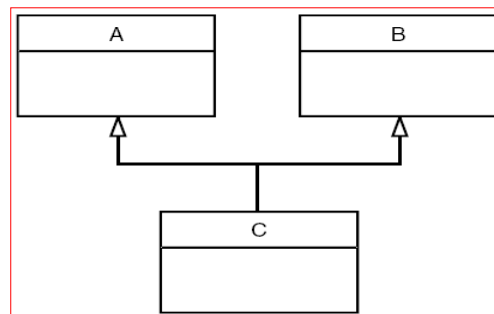
55

Virtual Destructors

<pre> class Base { public: virtual ~Base() { cout<<" Base Destroyed \n"; } }; class Derv: public Base { public: ~Derv() { cout<<" Derv Destroyed \n"; } }; </pre>	<pre> void main() { Base *p = new Derv; delete p; /*will call destructor of derived class and then base class*/ } </pre>
--	---

56

Multiple Inheritance



57

Multiple Inheritance – Syntax

- The base classes from which C is derived are listed following the colon in C's specification and are separated by commas

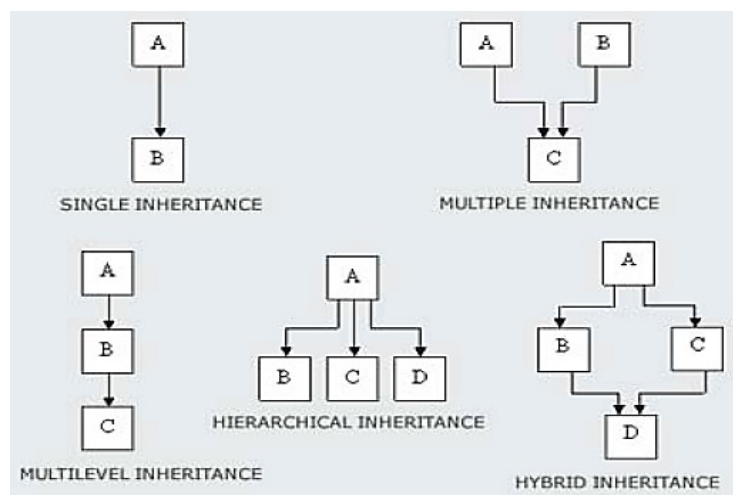
```
class A
{
};

class B
{
};

class C : public A, public B
{
};
```

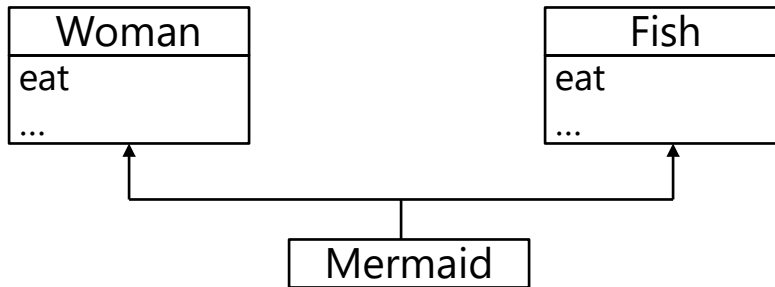
58

Types of Inheritance



64

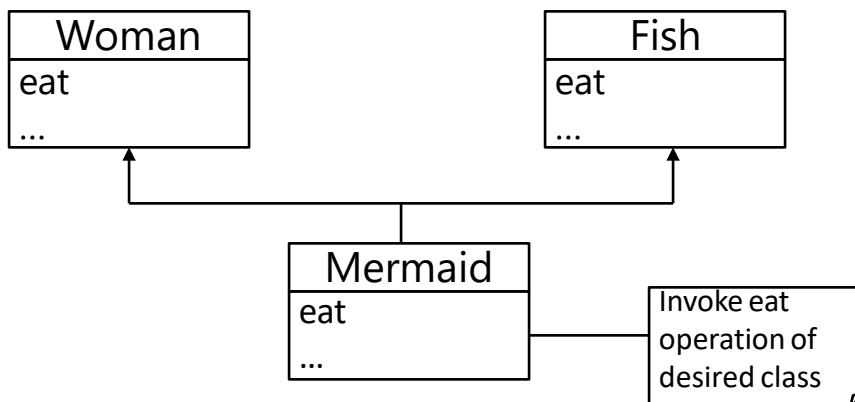
Multiple Inheritance Problems – Duplicate Features



- Which **eat** operation **Mermaid** inherits?

65

Solution – Override the common feature



66

Example

```
#include <iostream>
using namespace std;
```

```
class Base1 {
public:
    void show() { cout << "Base1 show()" << endl; }
};

class Base2 {
public:
    void show() { cout << "Base2 show()" << endl; }
};

class Derived : public Base1, public Base2 {
public:
    void display() {
        cout << "Derived display()" << endl;
    }
};
```

```
Derived d;
d.show(); //error
```

```
Derived d;

d.Base1::show(); // Calls show() from
Base1
d.Base2::show(); // Calls show() from
Base2
```

67

Example

```
class Base1 {
public:
    void show() { cout << "Base1 show()" << endl; }
};

class Base2 {
public:
    void show() { cout << "Base2 show()" << endl; }
};

class Derived : public Base1, public Base2 {
public:
    void display() {
        cout << "Derived display()" << endl;

        Base1::show(); // Call Base1's show()
        Base2::show(); // Call Base2's show()
    }
};
```

```
int main() {
    Derived d;
    d.display();
}
```

```
Output:
Derived display()
Base1 show()
Base2 show()
```

68